Fine-tune a transformer model (BERT/DistilBERT) to perform: Part-of-Speech (POS) Tagging- Chunking

1. Dataset Selection Option 1: Universal Dependencies (POS Tagging)

Used for Part-of-Speech tagging.

Common POS Labels:

NOUN VERB ADJ ADV PRON DET ADP NUM CONJ PRT PUNCT X Option 2: CoNLL-2003 (Chunking)

Used for Phrase Chunking.

Chunk Labels:

B-NP → Beginning of Noun Phrase I-NP → Inside Noun Phrase B-VP → Beginning of Verb Phrase I-VP → Inside Verb Phrase B-PP → Beginning of Preposition Phrase I-PP → Inside Preposition Phrase O → Outside any chunk

Deliverable Example:

Dataset Name: CoNLL-2003 Task: Chunking Labels: B-NP, I-NP, B-VP, I-VP, B-PP, I-PP, O

2. Data Preprocessing Steps: Load dataset Tokenize using BERT tokenizer Align labels with tokens Handle subwords Use -100 for special tokens Label Alignment Logic

When a word splits into subwords:

playing → play ##ing VERB → VERB VERB

```
!pip install transformers datasets seqeval evaluate -q
```

```
from datasets import load_dataset

dataset = load_dataset("conll2003")
dataset
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: Userw
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access publ
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/datasets/load.py:1486: FutureWarning: Th
You can avoid this message in future by passing the argument `trust_remote_code=
Passing `trust_remote_code=True` will be mandatory to load this dataset from the
```

```
  warnings.warn(
DatasetDict({
    train: Dataset({
        features: ['id', 'tokens', 'pos_tags', 'chunk_tags', 'ner_tags'],
        num_rows: 14041
    })
    validation: Dataset({
        features: ['id', 'tokens', 'pos_tags', 'chunk_tags', 'ner_tags'],
        num_rows: 3250
    })
    test: Dataset({
        features: ['id', 'tokens', 'pos_tags', 'chunk_tags', 'ner_tags'],
        num_rows: 3453
    })
})
```

```
from huggingface_hub import login
login("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")
```

```
import os
os.environ["HF_TOKEN"] = "hf_xxxxxxxxxxxxxxxxxxxxx"
```

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

label_list = dataset["train"].features["ner_tags"].feature.names
```

```
def tokenize_and_align_labels(examples):
    tokenized_inputs = tokenizer(
        examples["tokens"],
        truncation=True,
        padding="max_length",
        max_length=128,   # Reduce length here
        is_split_into_words=True
    )

    labels = []

    for i in range(len(examples["tokens"])):
        word_ids = tokenized_inputs.word_ids(batch_index=i)
        label_ids = []
        previous_word_idx = None

        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(-100)
            elif word_idx != previous_word_idx:
                label_ids.append(examples["ner_tags"][i][word_idx])
```

```
            else:
                label_ids.append(examples["ner_tags"][i][word_idx])

            previous_word_idx = word_idx

        labels.append(label_ids)

    tokenized_inputs["labels"] = labels
    return tokenized_inputs
```

```
tokenized_dataset = dataset.map(
    tokenize_and_align_labels,
    batched=True
)
```

Map: 100% 3250/3250 [00:02<00:00, 1299.04 examples/s]

```
tokenized_dataset
```

```
DatasetDict({
    train: Dataset({
        features: ['id', 'tokens', 'pos_tags', 'chunk_tags', 'ner_tags',
'input_ids', 'token_type_ids', 'attention_mask', 'labels'],
        num_rows: 14041
    })
    validation: Dataset({
        features: ['id', 'tokens', 'pos_tags', 'chunk_tags', 'ner_tags',
'input_ids', 'token_type_ids', 'attention_mask', 'labels'],
        num_rows: 3250
    })
    test: Dataset({
        features: ['id', 'tokens', 'pos_tags', 'chunk_tags', 'ner_tags',
'input_ids', 'token_type_ids', 'attention_mask', 'labels'],
        num_rows: 3453
    })
})
```

```
small_train_dataset = tokenized_dataset["train"].select(range(1000))
small_eval_dataset = tokenized_dataset["validation"].select(range(300))
```

```
from transformers import AutoModelForTokenClassification

model = AutoModelForTokenClassification.from_pretrained(
    "bert-base-cased",
    num_labels=len(label_list)
)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
```

Loading weights: 100% 197/197 [00:00<00:00, 2579.31it/s]

```
BertForTokenClassification LOAD REPORT from: bert-base-cased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
bert.pooler.dense.bias                       | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
bert.pooler.dense.weight                     | UNEXPECTED |
classifier.bias                              | MISSING    |
classifier.weight                            | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; no
- MISSING:  those params were newly initialized because missing from the checkpo
```

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_steps=50,
    save_steps=500
)
```

```
from transformers import DataCollatorForTokenClassification

data_collator = DataCollatorForTokenClassification(tokenizer)
```

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train_dataset,
    eval_dataset=small_eval_dataset,
    data_collator=data_collator
)
```

```
trainer.train()
```

[375/375 1:05:54, Epoch 3/3]

Step	Training Loss
50	0.890604
100	0.405116
150	0.292391
200	0.227569
250	0.159589
300	0.132604
350	0.130264

Writing model shards: 100% 1/1 [00:04<00:00, 4.01s/it]

```
TrainOutput(global_step=375, training_loss=0.306050807317098, metrics=
{'train_runtime': 3968.69, 'train_samples_per_second': 0.756,
'train_steps_per_second': 0.094, 'total_flos': 195984969984000.0, 'train_loss':
0.306050807317098, 'epoch': 3.0})
```

```
import numpy as np
import evaluate

metric = evaluate.load("seqeval")

predictions, labels, _ = trainer.predict(small_eval_dataset)
predictions = np.argmax(predictions, axis=2)

true_labels = [
    [label_list[l] for l in label if l != -100]
    for label in labels
]

true_predictions = [
    [label_list[p] for (p, l) in zip(pred, label) if l != -100]
    for pred, label in zip(predictions, labels)
]

results = metric.compute(
    predictions=true_predictions,
    references=true_labels
)

print("Precision:", results["overall_precision"])
print("Recall:", results["overall_recall"])
print("F1 Score:", results["overall_f1"])
```

```
Downloading builder script:        6.34k/? [00:00<00:00, 113kB/s]

Precision: 0.7994791666666666
Recall: 0.8418647166361974
F1 Score: 0.8201246660730186
```

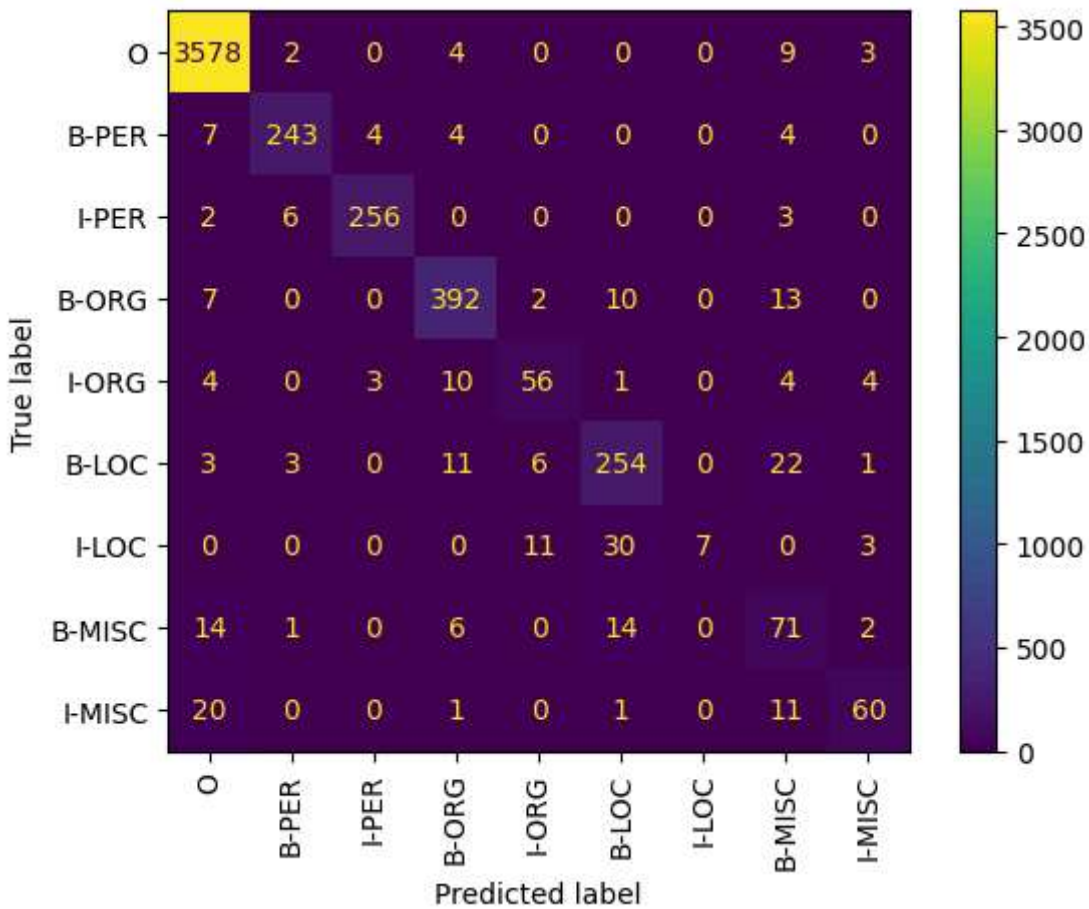
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

true_labels_flat = []
true_predictions_flat = []

for pred, lab in zip(predictions, labels):
    for p, l in zip(pred, lab):
        if l != -100:
            true_labels_flat.append(l)
            true_predictions_flat.append(p)

cm = confusion_matrix(true_labels_flat, true_predictions_flat)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_list)
disp.plot(xticks_rotation=90)
plt.show()
```



```
from sklearn.metrics import classification_report

print(classification_report(
    true_labels_flat,
    true_predictions_flat,
    target_names=label_list
))
```

```
              precision    recall  f1-score   support

           O       0.98      0.99      0.99      3596
       B-PER       0.95      0.93      0.94       262
       I-PER       0.97      0.96      0.97       267
       B-ORG       0.92      0.92      0.92       424
       I-ORG       0.75      0.68      0.71        82
       B-LOC       0.82      0.85      0.83       300
       I-LOC       1.00      0.14      0.24        51
      B-MISC       0.52      0.66      0.58       108
      I-MISC       0.82      0.65      0.72        93

    accuracy                           0.95      5183
   macro avg       0.86      0.75      0.77      5183
weighted avg       0.95      0.95      0.95      5183
```

```
from transformers import pipeline

ner_pipeline = pipeline(
    "token-classification",
    model=model,
    tokenizer=tokenizer
)

print(ner_pipeline("John works at Google in California"))
```

```
[{'entity': 'LABEL_1', 'score': np.float32(0.9215594), 'index': 1, 'word': 'John
```

Start coding or generate with AI.

```
from transformers import pipeline

ner_pipeline = pipeline(
    "token-classification",
    model=model,
    tokenizer=tokenizer
)

sentence = "John works at Google in California"
result = ner_pipeline(sentence)
```

```
for r in result:
    print(r["word"], "->", r["entity"])
```

```
John -> LABEL_1
works -> LABEL_0
at -> LABEL_0
Google -> LABEL_3
in -> LABEL_0
California -> LABEL_5
```

```
model.save_pretrained("bert-ner-model")
tokenizer.save_pretrained("bert-ner-model")
```

Writing model shards: 100% 1/1 [00:11<00:00, 11.32s/it]

```
('bert-ner-model/tokenizer_config.json', 'bert-ner-model/tokenizer.json')
```

Start coding or generate with AI.